

Data Mining HW#1 – 40435056

Razie Doregard

Subject: KNN Classifier

Stage#1: Loading the data

Data head:

```
age      51
sex      0
cp       0
trestbps 51
chol     13
fbs      0
restecg  0
thalach  17
exang    0
oldpeak  1
slope    0
ca       51
thal     51
target   0
dtype: int64
```

Checking the missing values of each attribute:

```
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  \
0  52.0   1   0    125.0  212.0   0         1    168.0     0     1.0
1  53.0   1   0    140.0  203.0   1         0    155.0     1     3.1
2  70.0   1   0    145.0  174.0   0         1    125.0     1     2.6
3  61.0   1   0    148.0  203.0   0         1    161.0     0     0.0
4  62.0   0   0    138.0  294.0   1         1    106.0     0     1.9

   slope  ca  thal  target
0      2  2.0   3.0      0
1      0  0.0   3.0      0
2      0  0.0   3.0      0
3      2  1.0   3.0      0
4      1  NaN   NaN      0
```

Stage#2: Handling Noisy Values

KNN Imputation to fill out missing values:

KNN imputation (K-Nearest Neighbors imputation) is a method to fill in missing values in your dataset by looking at the most similar (nearest) rows in the data.

Instead of replacing missing values with a simple mean or median, KNN imputation uses information from other data points that are most similar to the one with missing data.

IQR method to detect outliers:

The Interquartile Range (IQR) measures the spread of the middle 50% of a dataset.

It's computed as:

$$\text{IQR} = Q3 - Q1$$

$$\text{IQR} = Q3 - Q1$$

where:

Q1 = 25th percentile (the value below which 25% of data fall)

Q3 = 75th percentile (the value below which 75% of data fall)

So, the IQR tells you how spread out the central data is, ignoring extremes.

```
def remove_outliers_iqr(df, exclude_cols=[]):
    df_clean = df.copy()
    numeric_cols = df_clean.select_dtypes(include=[np.number]).columns

    for col in numeric_cols:
        if col in exclude_cols:
            continue

        Q1 = df_clean[col].quantile(0.25)
        Q3 = df_clean[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 3 * IQR
        upper_bound = Q3 + 3 * IQR

        before = len(df_clean)
        df_clean = df_clean[(df_clean[col] >= lower_bound) & (df_clean[col] <= upper_bound)]
        after = len(df_clean)

        print(f"{col}: removed {before - after} outliers")

    return df_clean
```

Stage#3: Handling Missing Values

Attributes' Missing Values:

- 1- Numeric: fill out the missing values with mean
- 2- Non-Numeric: fill out the missing values with mode

```
def fill_missing_values(df: pd.DataFrame) -> pd.DataFrame:
    df_filled = df.copy()

    for column in df_filled.columns:
        if df_filled[column].dtype in [np.float64, np.int64]:
            # numeric: mean
            mean_value = df_filled[column].mean()
            df_filled[column].fillna(mean_value, inplace=True)
        else:
            # categorical: mode
            mode_value = df_filled[column].mode()
            if not mode_value.empty:
                df_filled[column].fillna(mode_value[0], inplace=True)

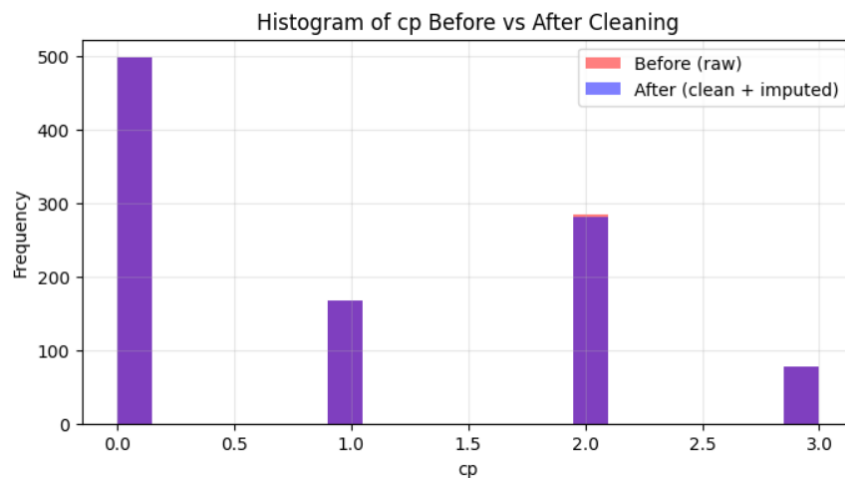
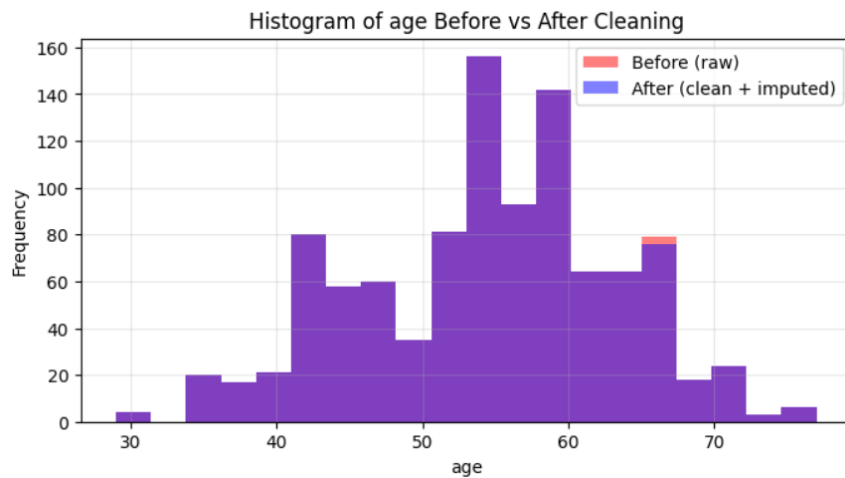
    return df_filled
```

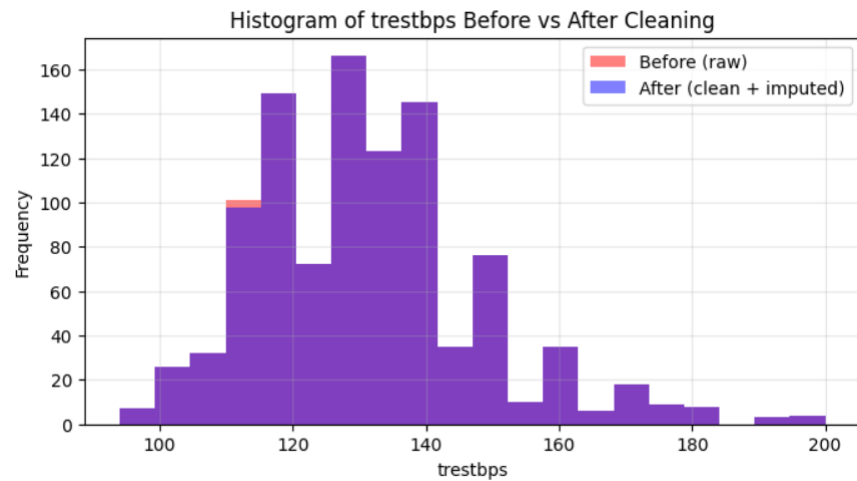
Result after removing the noises and the missing the values:

```
age: removed 0 outliers
cp: removed 0 outliers
trestbps: removed 0 outliers
chol: removed 3 outliers
restecg: removed 0 outliers
thalach: removed 0 outliers
oldpeak: removed 0 outliers
slope: removed 0 outliers
ca: removed 0 outliers
thal: removed 0 outliers
Missing values after imputation:
age          0
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           0
thal         0
target       0
dtype: int64
```

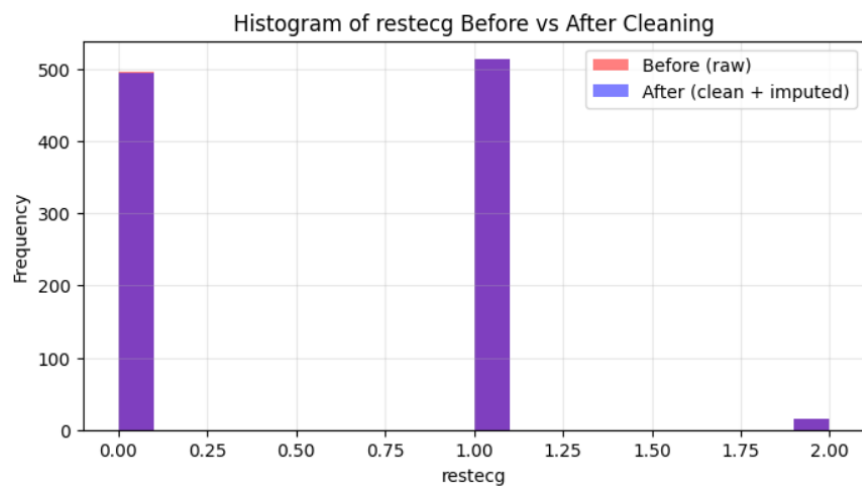
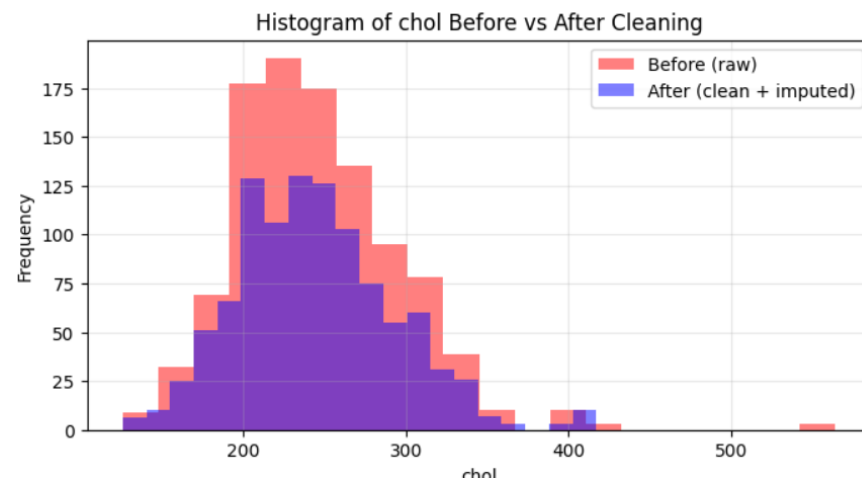
Stage#4: Plotting the histograms of the attributes

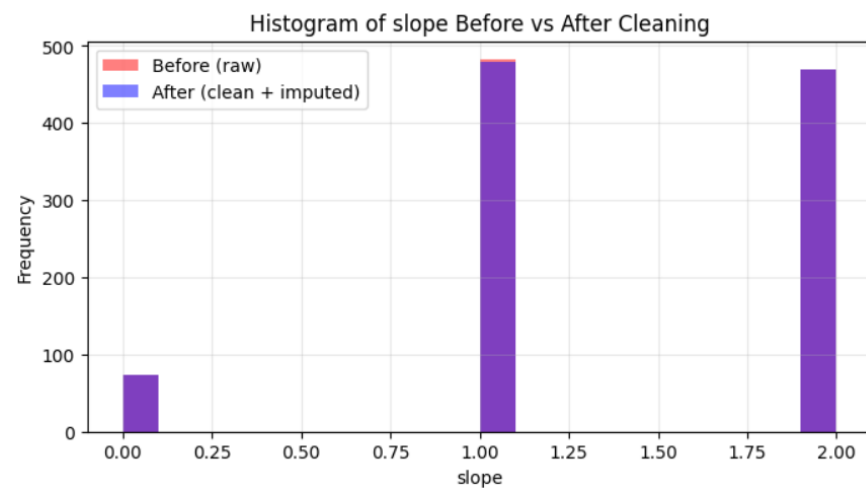
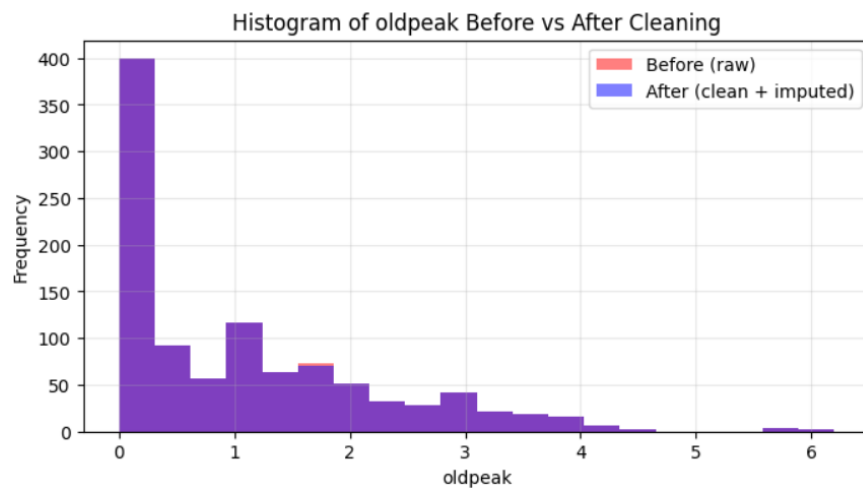
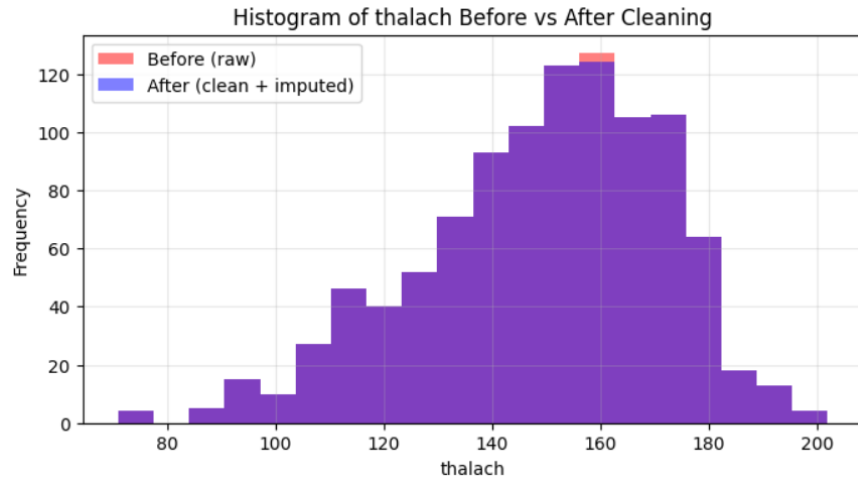
As can be seen from the histograms before and after the preprocessing phase, the attributes with outliers showed a larger difference in their histograms. The attributes having the largest number of outliers are chol and thalach and after the cleaning, their histogram became a little bit more like the normal distribution which is totally desired. The rest of attributes did not contain many outliers and therefore, their distribution did not change much.

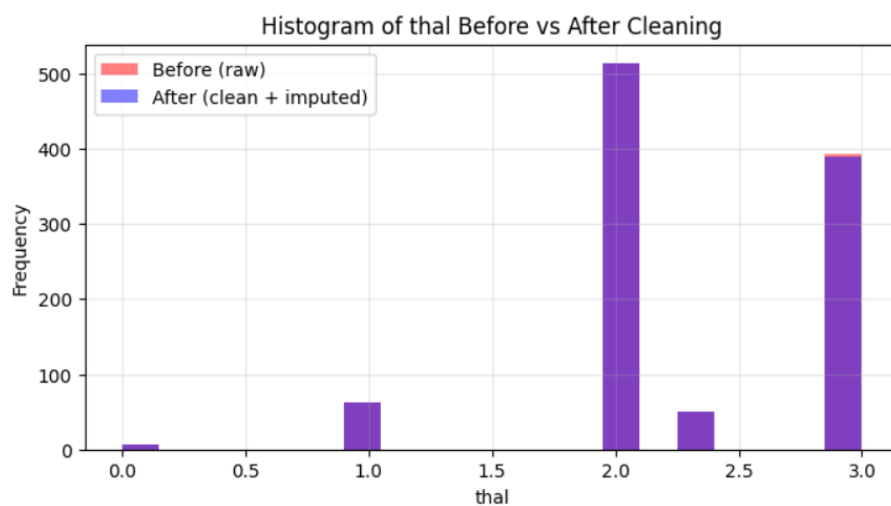
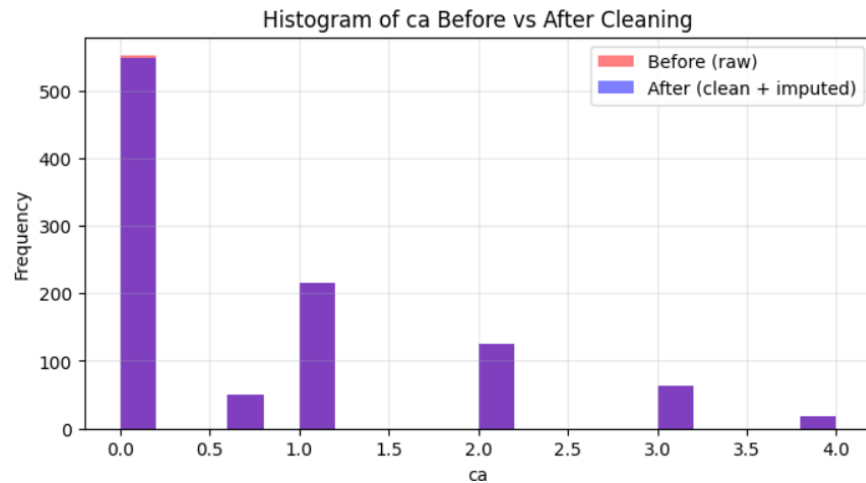




Chol attribute had the most missing values which were filled after the preprocessing phase.







Stage#5: Normalization

The standard score or z-score is the number of standard deviations by which the value of a raw score is above or below the mean value of what is being observed or measured. Raw scores above the mean have positive standard scores, while those below the mean have negative standard scores.

The normalization of numeric columns was done with z-score in this part:

```
df = df_imputed
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
df_scaled = df.copy()
df_scaled[numeric_cols] = (df[numeric_cols] - df[numeric_cols].mean()) / df[numeric_cols].std()
print(df_scaled.describe())
```

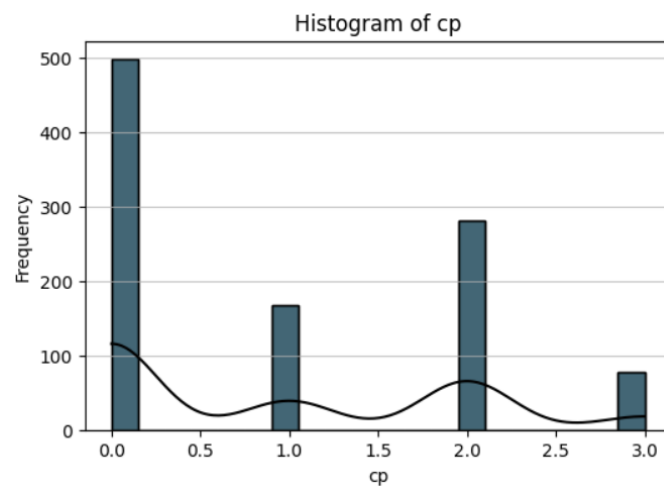
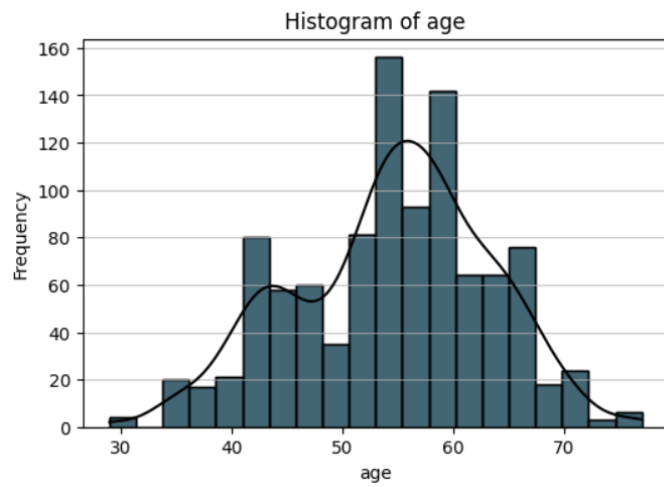

After applying z-score, the mean is almost 0 and std is 1 which is expected after z-score normalization:

	age	sex	cp	trestbps	chol
count	1.022000e+03	1.022000e+03	1.022000e+03	1.022000e+03	1.022000e+03
mean	-8.899165e-16	-5.561978e-17	3.476236e-17	1.025490e-16	-4.866731e-17
std	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00
min	-2.889991e+00	-1.518284e+00	-9.123714e-01	-2.198728e+00	-2.460638e+00
25%	-7.312043e-01	-1.518284e+00	-9.123714e-01	-6.764516e-01	-7.032757e-01
50%	6.413810e-02	6.579939e-01	5.892399e-02	-9.096085e-02	-1.037049e-01
75%	6.322398e-01	6.579939e-01	1.030219e+00	4.945299e-01	5.992402e-01
max	2.563786e+00	6.579939e-01	2.001515e+00	4.007474e+00	3.555744e+00

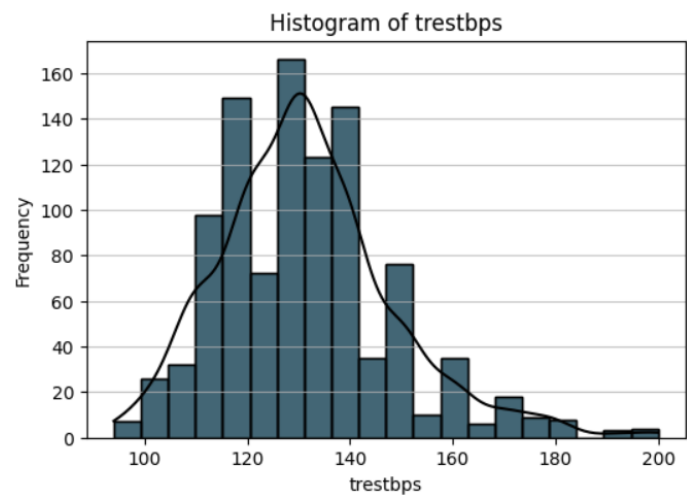
	fbs	restecg	thalach	exang	oldpeak
count	1.022000e+03	1.022000e+03	1.022000e+03	1.022000e+03	1.022000e+03
mean	2.085742e-17	2.780989e-17	-1.484353e-15	3.650048e-17	-1.460019e-16
std	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00
min	-4.193950e-01	-1.006519e+00	-3.440126e+00	-7.135142e-01	-9.103106e-01
25%	-4.193950e-01	-1.006519e+00	-7.153139e-01	-7.135142e-01	-9.103106e-01
50%	-4.193950e-01	8.878873e-01	1.197092e-01	-7.135142e-01	-2.275995e-01
75%	-4.193950e-01	8.878873e-01	7.349894e-01	1.400142e+00	6.257893e-01
max	2.382054e+00	2.782294e+00	2.317139e+00	1.400142e+00	4.380700e+00

	slope	ca	thal	target
count	1.022000e+03	1.022000e+03	1.022000e+03	1.022000e+03
mean	9.559650e-17	-6.952473e-17	1.286207e-16	-1.042871e-17
std	1.000000e+00	1.000000e+00	1.000000e+00	1.000000e+00
min	-2.242404e+00	-7.482107e-01	-3.846700e+00	-1.023265e+00
25%	-6.250879e-01	-7.482107e-01	-5.384810e-01	-1.023265e+00
50%	-6.250879e-01	-7.482107e-01	-5.384810e-01	9.763080e-01
75%	9.922281e-01	2.485304e-01	1.115628e+00	9.763080e-01
max	9.922281e-01	3.238754e+00	1.115628e+00	9.763080e-01

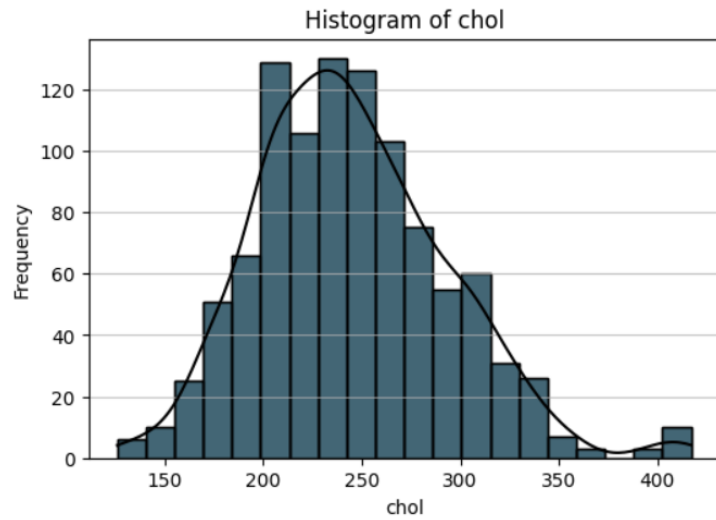
Checking the normality of attributes using histograms:



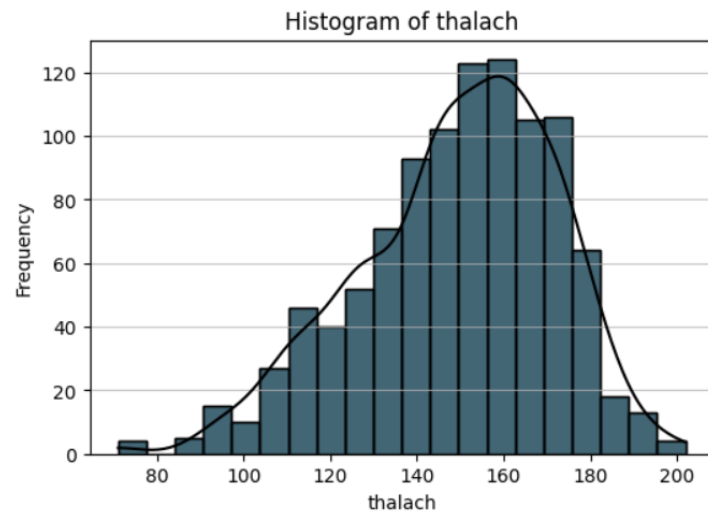
Trestbps: right skewness



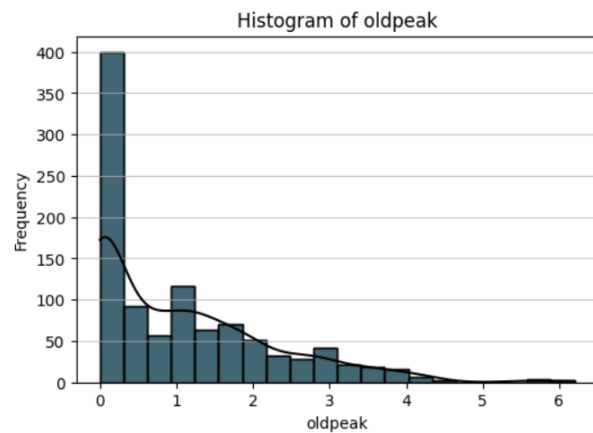
Chol: a little bit of right skewness



Thalach: left skewness



Oldpeak: right skewness

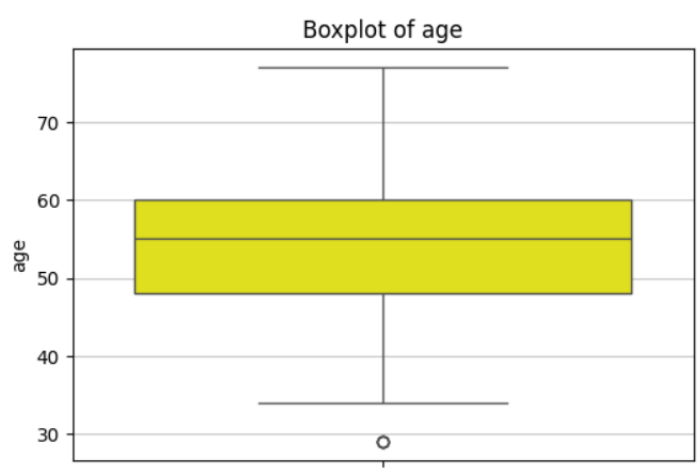


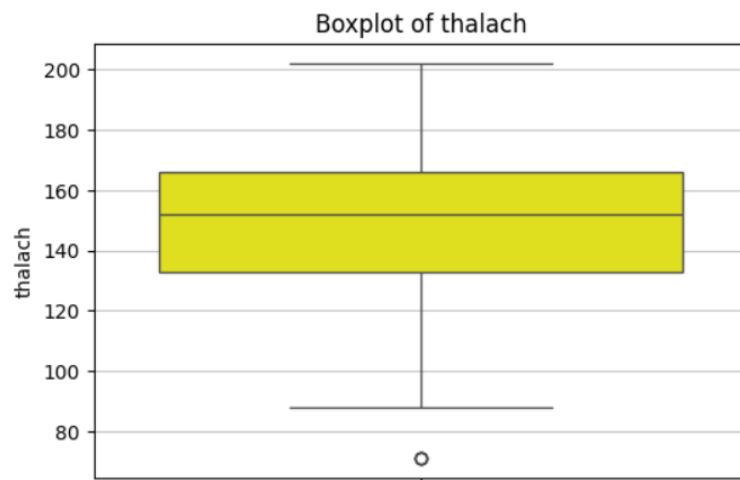
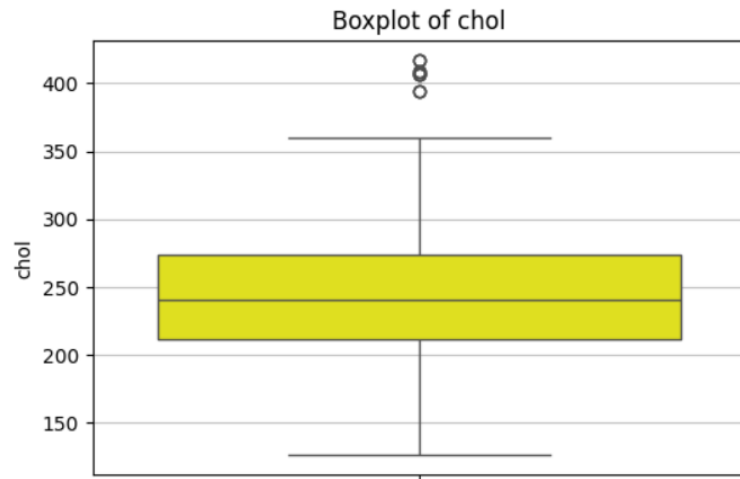
Stage#6: Boxplots of the attributes

A box plot uses boxes and lines to depict the distributions of one or more groups of numeric data. Box limits indicate the range of the central 50% of the data, with a central line marking the median value.

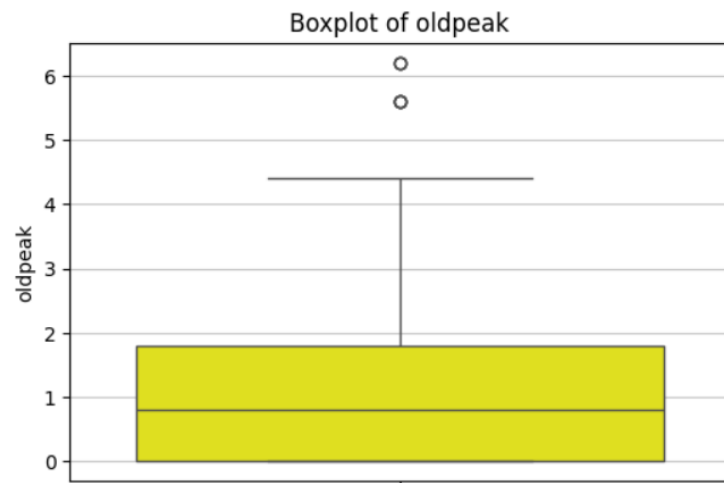
Boxplot cases:

- 1- The median is close to the **top edge** Q3 → The lower half (Q1 → median) is **larger**
- 2- The median is roughly in the **middle** of the box → Distance from Q1→median $\approx$ Q3→median.
- 3- The median is closer to the **bottom edge** Q1 → The upper half of the box (median → Q3) is **larger**.





The first 25% of the data is under 0 and cannot be seen in the current boxplot for oldpeak:



Stage#7: Q-Q plots of attributes

A QQ plot is a scatterplot created by plotting two sets of quantiles against one another. If both sets of quantiles came from the same distribution, we should see the points forming a line that's roughly straight.

1. Right-Skewed Distribution (Positive Skew)

What the Q–Q plot looks like:

- Points follow the line in the middle.
- But at the **right end**, points **curve upward above the line**.
- At the **left end**, points **dip below the line**.

Symmetric (No Skew)

What the Q–Q plot looks like:

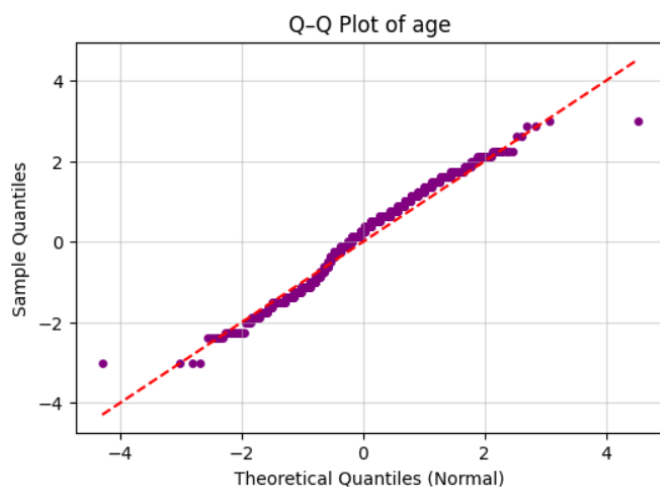
- Points lie **close to the diagonal** across the entire plot.
- Slight random scatter is fine.

2. Left-Skewed Distribution (Negative Skew)

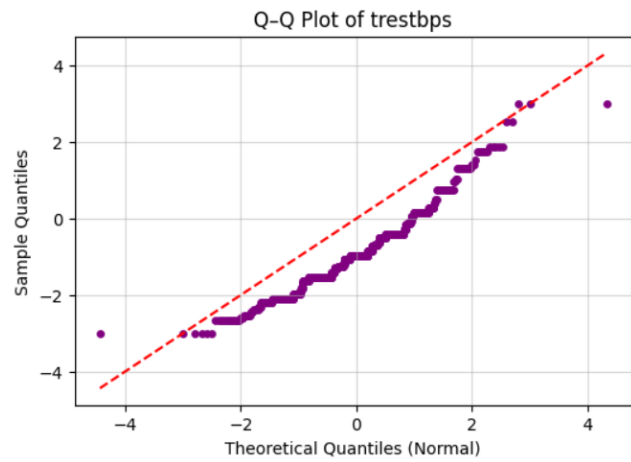
What the Q–Q plot looks like:

- Points follow the line in the middle.
- But at the **right end**, points **curve below the line**.
- At the **left end**, points **rise above the line**.

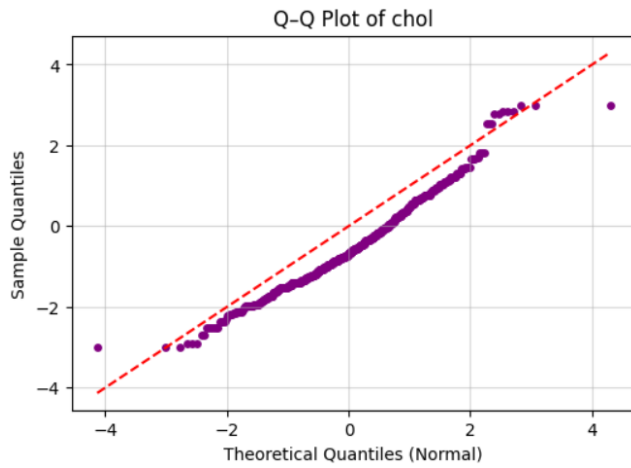
Almost normal:



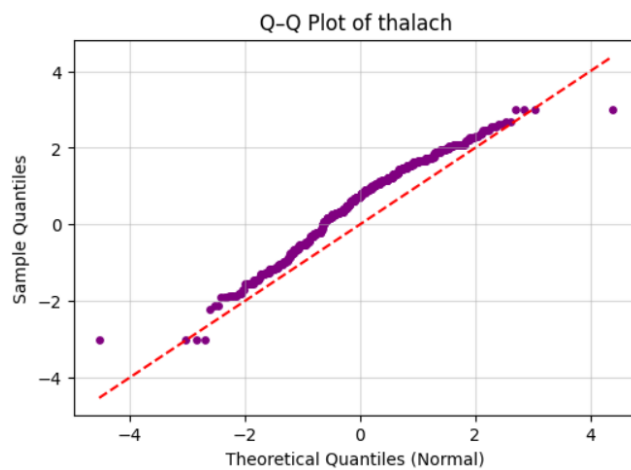
Left-Skewed Distribution:



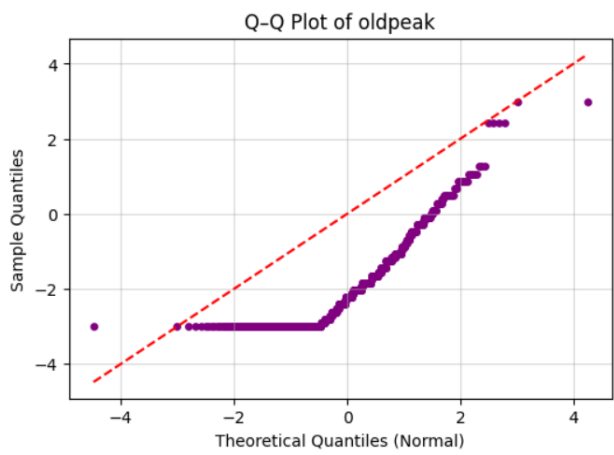
Left-Skewed Distribution:



Right-Skewed Distribution:



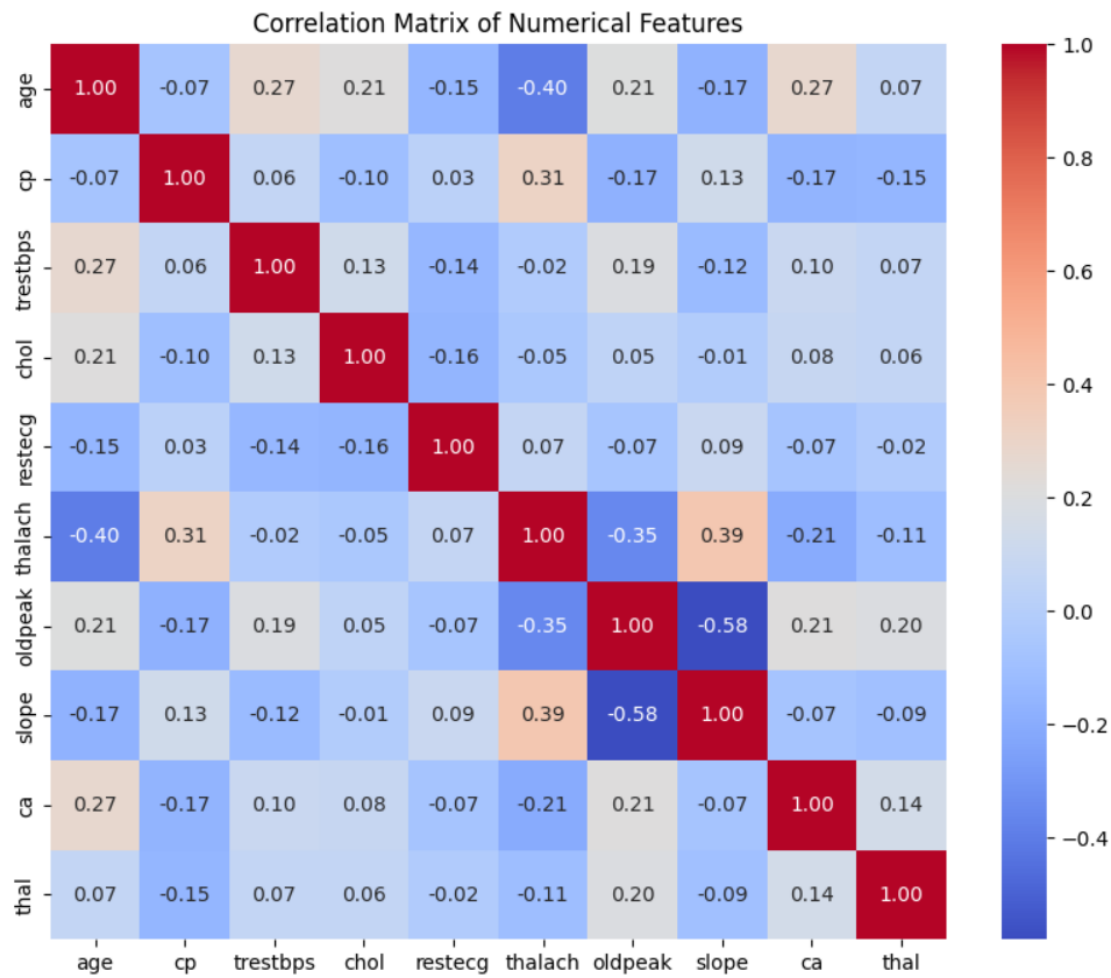
Left-Skewed Distribution:



Stage#8: Correlation Matrix

Most features are reasonably independent, which is good for KNN — each contributes differently to distance calculations.

However, a tiny bit of correlation can be seen between slope (Slope of the peak exercise ST segment (0–2)) and oldpeak (ST depression induced by exercise relative to rest).



Stage#9: Splitting train and test data 80 - 20

```
x = df.drop('target', axis=1).to_numpy()
y = df['target'].to_numpy()

np.random.seed(42)
indices = np.arange(len(df))
np.random.shuffle(indices)

train_size = int(0.8 * len(df))
train_indices = indices[:train_size]
test_indices = indices[train_size:]

x_train, y_train = x[train_indices], y[train_indices]
x_test, y_test = x[test_indices], y[test_indices]
```

Stage#10: implementation of KNN

The distance is calculated by the Euclidean distance and weighted knn. In regular k-NN:

- You pick the k nearest neighbors.
- Each neighbor votes equally.

In weighted k-NN:

- Each neighbor votes proportionally to its closeness.
- Closer neighbor → higher weight
- Far neighbor → smaller weight

So the final prediction is based on weighted votes, not simple counts.

Formula:

$$w_i = \frac{1}{d_i + \epsilon}$$

```

def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def knn_predict_weighted(X_train, y_train, X_test, k=5):
    y_pred = []
    for test_point in X_test:
        distances = np.array([euclidean_distance(test_point, x) for x in X_train])
        nn_indices = distances.argsort()[:k]
        nn_labels = y_train[nn_indices]
        nn_distances = distances[nn_indices]
        weights = 1 / (nn_distances + 1e-5)
        label_weights = {}
        for label, w in zip(nn_labels, weights):
            label_weights[label] = label_weights.get(label, 0) + w
        y_pred.append(max(label_weights, key=label_weights.get))
    return np.array(y_pred)

best_k = 1
best_acc = 0
for k in range(1, 21, 2):
    y_pred = knn_predict_weighted(X_train, y_train, X_test, k=k)
    acc = np.mean(y_pred == y_test)
    print(f"k={k}, accuracy={acc:.3f}")
    if acc > best_acc:
        best_acc = acc
        best_k = k
print("Best k:", best_k, "with accuracy:", best_acc)

```

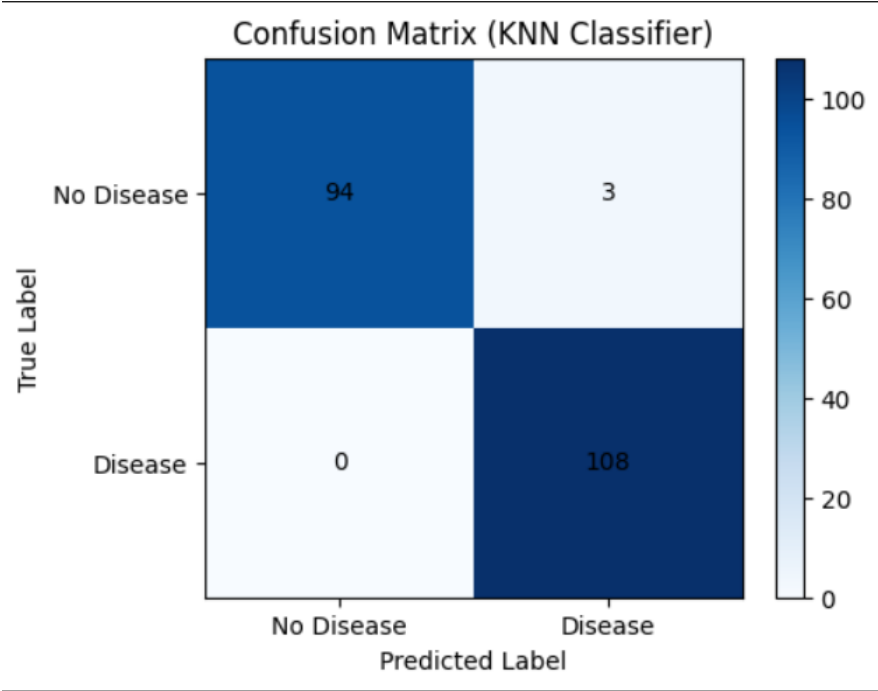
Accuracy Results: the best k was detected at 17

```

k=1, accuracy=0.961
k=3, accuracy=0.956
k=5, accuracy=0.956
k=7, accuracy=0.971
k=9, accuracy=0.976
k=11, accuracy=0.980
k=13, accuracy=0.980
k=15, accuracy=0.980
k=17, accuracy=0.985
k=19, accuracy=0.971
Best k: 17 with accuracy: 0.9853658536585366

```

Confusion Matrix



Scores after applying k-fold cross variation:

	k	Accuracy	Precision	Recall	F1-score
0	1	0.942	0.946	0.940	0.943
1	3	0.928	0.915	0.945	0.929
2	5	0.939	0.932	0.946	0.938
3	7	0.941	0.936	0.949	0.942
4	9	0.923	0.923	0.929	0.924
5	11	0.929	0.932	0.932	0.930
6	13	0.934	0.927	0.943	0.935
7	15	0.920	0.919	0.925	0.922
8	17	0.939	0.942	0.939	0.940
9	19	0.935	0.931	0.945	0.937

Scores explained:

Precision: Out of all the actual positive cases, how many did your model correctly detect:
How good your model is at finding all the positives

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Out of all the actual positive cases, how many did your model correctly detect:
How good your model is at finding all the positives.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Accuracy: Out of all predictions made, how many were correct

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

F1 Score: The harmonic mean of precision and recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Stage#11: K-fold variation

```
def k_fold_cross_validation(X, y, k_neighbors=5, n_splits=5, weighted=True):
    fold_size = len(X) // n_splits
    indices = np.arange(len(X))
    np.random.shuffle(indices)

    accuracies, precisions, recalls, f1s = [], [], [], []

    for i in range(n_splits):
        test_idx = indices[i*fold_size:(i+1)*fold_size]
        train_idx = np.setdiff1d(indices, test_idx)

        X_train, y_train = X[train_idx], y[train_idx]
        X_test, y_test = X[test_idx], y[test_idx]

        if weighted:
            y_pred = knn_predict_weighted(X_train, y_train, X_test, k=k_neighbors)
        else:
            y_pred = knn_predict(X_train, y_train, X_test, k=k_neighbors)

        acc, prec, rec, f1 = compute_metrics(y_test, y_pred)
        accuracies.append(acc)
        precisions.append(prec)
        recalls.append(rec)
        f1s.append(f1)

    return {
        'Accuracy': np.mean(accuracies),
        'Precision': np.mean(precisions),
        'Recall': np.mean(recalls),
        'F1-score': np.mean(f1s)
    }

k_values = range(1, 21, 2)
rows = []

for k in k_values:
```

Elbow method:

The elbow method is a technique used to choose the best number of clusters (k) in k-means clustering.

It works by checking:

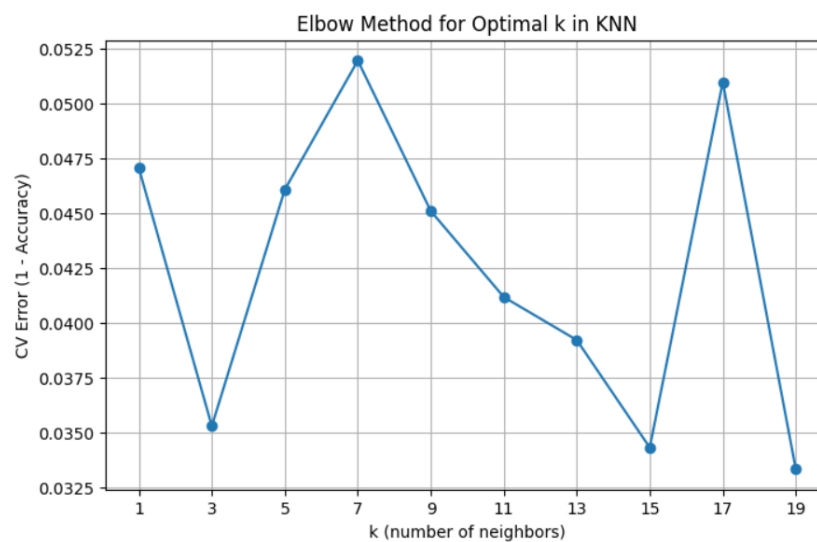
- how the within-cluster error decreases as you increase k,
- and finding the point where adding more clusters doesn't improve things much anymore.

This point looks like an elbow on a graph.

```
def knn_error_for_k(X, y, k_values, n_splits=5, weighted=True):
    errors = []
    for k in k_values:
        metrics = k_fold_cross_validation(X, y, k_neighbors=k, n_splits=n_splits, weighted=weighted)
        error = 1 - metrics['Accuracy']
        errors.append(error)
        print(f"k={k}, CV Error={error:.3f}")
    return errors

k_values = range(1, 21, 2)
cv_errors = knn_error_for_k(X, y, k_values, n_splits=5)

plt.figure(figsize=(8,5))
plt.plot(k_values, cv_errors, marker='o')
plt.xlabel('k (number of neighbors)')
plt.ylabel('CV Error (1 - Accuracy)')
plt.title('Elbow Method for Optimal k in KNN')
plt.xticks(k_values)
plt.grid(True)
plt.show()
```



Accuracy results:

```
k=1, CV Error=0.047  
k=3, CV Error=0.035  
k=5, CV Error=0.046  
k=7, CV Error=0.052  
k=9, CV Error=0.045  
k=11, CV Error=0.041  
k=13, CV Error=0.039  
k=15, CV Error=0.034  
k=17, CV Error=0.051  
k=19, CV Error=0.033
```

Based on the elbow method the best amounts of k are 15 and 3 respectively.